ORBIT · PYTHON · PAPER 06

第 6 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

关于列表的可变性，以下说法正确的是：

A. 列表是不可变对象
B. 列表是可变对象，可以修改元素
C. 列表不能添加或删除元素
D. 列表和元组完全相同

答案 B. 列表是可变对象，可以修改元素

解析 列表是Python中的可变对象，可以修改、添加、删除元素。

示例：

```
lst = [1, 2, 3]
print(f"原始列表: {lst}")
# 修改元素
lst[0] = 10
print(f"修改后: {lst}")
# [10, 2, 3]
# 添加元素
lst.append(4)
print(f"添加后: {lst}")
# [10, 2, 3, 4]
# 删除元素
lst.remove(2)
print(f"删除后: {lst}")
# [10, 3, 4]
知识拓展：可变对象 vs 不可变对象
# 可变对象：list, dict, set
mutable_list = [1, 2, 3]
print(id(mutable_list))
mutable_list.append(4)
print(id(mutable_list))
# ID不变
# 不可变对象：int, str, tuple
immutable_str = "hello"
print(id(immutable_str))
immutable_str = immutable_str + " world"
print(id(immutable_str))
# ID改变，创建了新对象
```

02 • 单项选择

以下代码的输出是什么？

```
list1 = [1, 2, 3]
list2 = list1
list2.append(4)
print(list1)
```

A. [1, 2, 3]
B. [1, 2, 3, 4]
C. [4]

D. 报错

答案 B. [1, 2, 3, 4]

解析 list2 = list1 创建的是引用，两个变量指向同一个对象，修改list2会影响list1。

示例：

```
list1 = [1, 2, 3]
list2 = list1
# 引用赋值
list2.append(4)
print(list1)
# [1, 2, 3, 4]
print(list2)
# [1, 2, 3, 4]
print(id(list1) == id(list2))
# True
```

知识拓展：引用 vs 拷贝

```
# 引用赋值
a = [1, 2, 3]
b = a
print(a is b)
# True
# 浅拷贝
c = a.copy()
print(a is c)
# False
c.append(4)
print(a)
# [1, 2, 3]
print(c)
# [1, 2, 3, 4]
# 切片也是浅拷贝
d = a[:]
print(a is d)
# False
```

03 • 单项选择

关于元组的不可变性，以下说法正确的是：

A. 元组的元素完全不能改变

B. 元组可以添加元素

C. 元组不可变是指元组对象本身不可变

D. 元组可以使用append()方法

答案 C. 元组不可变是指元组对象本身不可变

解析 元组的不可变性是指元组对象本身（元素的引用）不可变，但元素对象本身可以是可变的。

示例：

```
t = (1, 2, 3)
# t[0] = 10 # TypeError: 元组不支持修改
# 但元组中的可变对象可以修改
t_with_list = ([1, 2], 3)
t_with_list[0].append(3)
```

```
# 修改列表内容
print(t_with_list)
# ([1, 2, 3], 3)
# 不能修改元组的元素引用
# t_with_list[0] = [4, 5] # TypeError
知识拓展：元组的不可变性详解
# 元组的元素引用不能改变
t1 = (1, 2, 3)
print(id(t1[0]))
# t1[0] = 10 # 错误：不能改变引用
# 但如果元素是可变对象，对象内容可以改变
t2 = ([1, 2], [3, 4])
print(id(t2[0]))
# 列表的ID
t2[0].append(5)
# 修改列表内容
print(id(t2[0]))
# 列表的ID不变
print(t2)
# ([1, 2, 5], [3, 4])
```

04 • 单项选择

以下代码的输出是什么？

```
t = ([1, 2], [3, 4])
t[0].append(5)
print(t)
```

A. 报错

B. ([1, 2], [3, 4])

C. ([1, 2, 5], [3, 4])

D. ([1, 2], [3, 4, 5])

答案 C. ([1, 2, 5], [3, 4])

解析 元组中的列表是可变对象，可以修改列表的内容，但不能修改元组的元素引用。

```
示例：
t = ([1, 2], [3, 4])
t[0].append(5)
# 修改第一个列表
print(t)
# ([1, 2, 5], [3, 4])
# 不能替换整个列表
# t[0] = [7, 8] # TypeError
知识拓展：嵌套可变对象
# 元组包含可变对象
t = ([1, 2], {3: 4}, {5, 6})
# 可以修改列表
t[0].append(3)
print(t[0])
# [1, 2, 3]
# 可以修改字典
t[1][7] = 8
```

```
print(t[1])
# {3: 4, 7: 8}
# 可以修改集合
t[2].add(7)
print(t[2])
# {5, 6, 7}
# 但不能替换元组的元素
# t[0] = [] # TypeError
```

05 • 单项选择

关于可变对象作为函数默认参数，以下说法正确的是：

A. 完全没有问题
B. 可能导致意外的行为
C. 默认参数只在函数定义时计算一次
D. B和C都正确

答案 D. B和C都正确

解析 可变对象作为默认参数会导致意外行为，因为默认参数只在函数定义时计算一次。

示例：

```
def bad_append(item, lst=[]):
    lst.append(item)
    return lst
print(bad_append(1))
# [1]
print(bad_append(2))
# [1, 2] - 意外!
print(bad_append(3))
# [1, 2, 3] - 意外!
```

知识拓展：默认参数的正确使用

```
# 错误做法
def bad_function(x, lst=[]):
    lst.append(x)
    return lst
# 正确做法
def good_function(x, lst=None):
    if lst is None:
        lst = []
    lst.append(x)
    return lst
print(good_function(1))
# [1]
print(good_function(2))
# [2] - 正确!
# 查看默认参数对象
print(bad_function.__defaults__)
# ([1, 2, 3],)
```

06 • 单项选择

以下代码的输出是什么？

```
def append_to_list(value, lst=[]):
```

```
    lst.append(value)
    return lst
print(append_to_list(1))
print(append_to_list(2))
```

A. [1] [2]

B. [1] [1, 2]

C. [1, 2] [1, 2]

D. 报错

答案 B. [1] [1, 2]

解析 默认参数 [] 在函数定义时创建一次，两次调用共享同一个列表对象。

示例：

```
def append_to_list(value, lst=[]):
    lst.append(value)
    return lst
result1 = append_to_list(1)
print(result1)
# [1]
result2 = append_to_list(2)
print(result2)
# [1, 2]
print(result1 is result2)
                # True - 同一个对象
```

知识拓展：默认参数陷阱详解

```
# 问题根源：默认参数在函数定义时创建
def demo(x, lst=[]):
    lst.append(x)
    return lst
print(id(demo.__defaults__[0]))
# 默认列表的ID
result1 = demo(1)
print(id(result1))
# 相同的ID
result2 = demo(2)
print(id(result2))
# 相同的ID
# 解决方案
def safe_demo(x, lst=None):
    if lst is None:
        lst = []
    # 每次调用创建新列表
    lst.append(x)
    return lst
result3 = safe_demo(1)
result4 = safe_demo(2)
print(result3)
# [1]
print(result4)
# [2]
print(result3 is result4)
# False
```

07 • 单项选择

如何正确定义带有可变默认参数的函数？

A. 使用空列表[]作为默认值
B. 使用None作为默认值，在函数内部创建列表
C. 使用tuple作为默认值
D. 不能使用可变对象

答案 B. 使用None作为默认值，在函数内部创建列表

解析 使用None作为默认值，在函数内部判断并创建新对象。

示例：

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
print(add_item(1))
# [1]
print(add_item(2))
# [2]
print(add_item(3, [10, 20]))
# [10, 20, 3]
```

知识拓展：其他安全的默认参数模式

```
# 模式1: 使用None
def method1(x, lst=None):
    lst = lst if lst is not None else []
    lst.append(x)
    return lst
# 模式2: 使用不可变对象作为标记
def method2(x, lst=()):
    if not lst:
        lst = []
    else:
        lst = list(lst)
    # 转换为列表
    lst.append(x)
    return lst
# 模式3: 使用*args
def method3(x, *items):
    lst = list(items)
    lst.append(x)
    return lst
print(method3(1, 2, 3))
# [2, 3, 1]
```

08 • 单项选择

以下代码的输出是什么？

```
def modify_list(lst):
    lst.append(4)
my_list = [1, 2, 3]
```

```
modify_list(my_list)
print(my_list)
```

A. [1, 2, 3]
B. [1, 2, 3, 4]
C. [4]
D. 报错

答案 B. [1, 2, 3, 4]

解析 列表是可变对象，函数内部的修改会影响外部的列表。

示例：

```
def modify_list(lst):
    lst.append(4)
my_list = [1, 2, 3]
modify_list(my_list)
print(my_list)
# [1, 2, 3, 4]
知识拓展：可变对象在函数中的行为
# 修改可变对象的内容
def modify_content(lst):
    lst.append(100)
    lst[0] = 999
data = [1, 2, 3]
modify_content(data)
print(data)
# [999, 2, 3, 100]
# 重新赋值不影响外部
def reassign(lst):
    lst = [999]
    # 创建新对象
    print(f"函数内: {lst}")
data = [1, 2, 3]
reassign(data)
# 函数内: [999]
print(data)
# [1, 2, 3] - 外部不变
# 如果需要避免修改外部
def safe_modify(lst):
    lst = lst.copy()
    # 创建副本
    lst.append(100)
    return lst
data = [1, 2, 3]
result = safe_modify(data)
print(data)
# [1, 2, 3] - 不变
print(result)
# [1, 2, 3, 100]
```

09 • 单项选择

关于列表的浅拷贝和深拷贝，以下说法正确的是：

A. 浅拷贝创建新对象，但元素是引用

B. 深拷贝递归复制所有元素

C. list.copy()是浅拷贝

D. 以上都正确

答案 D. 以上都正确

解析 所有选项都正确：浅拷贝创建新对象但元素是引用，深拷贝递归复制所有元素，list.copy()是浅拷贝。

示例：

```
import copy
original = [[1, 2], [3, 4]]
# 浅拷贝
shallow = original.copy()
shallow[0].append(5)
print(original)
# [[1, 2, 5], [3, 4]] - 受影响
print(shallow)
# [[1, 2, 5], [3, 4]]
# 深拷贝
original = [[1, 2], [3, 4]]
deep = copy.deepcopy(original)
deep[0].append(5)
print(original)
# [[1, 2], [3, 4]] - 不受影响
print(deep)
# [[1, 2, 5], [3, 4]]
```

知识拓展：浅拷贝 vs 深拷贝详解

```
import copy
# 原始对象
original = [[1, 2], [3, 4]]
# 引用赋值
ref = original
ref[0].append(999)
print(original)
# [[1, 2, 999], [3, 4]]
# 浅拷贝的各种方式
original = [[1, 2], [3, 4]]
shallow1 = original.copy()
shallow2 = original[:]
shallow3 = list(original)
shallow4 = copy.copy(original)
# 浅拷贝：外层独立，内层共享
shallow1[0].append(5)
print(original)
# [[1, 2, 5], [3, 4]]
# 深拷贝：完全独立
original = [[1, 2], [3, 4]]
deep = copy.deepcopy(original)
deep[0].append(5)
print(original)
# [[1, 2], [3, 4]]
```

```
print(deep)
# [[1, 2, 5], [3, 4]]
```

10 • 单项选择

以下代码的输出是什么？

```
import copy
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
list2[0].append(5)
print(list1)
```

A. 1, 2], [3, 4

B. 1, 2, 5], [3, 4

C. 1, 2], [3, 4, 5

D. 报错

答案 B. 1, 2, 5], [3, 4

解析 list1.copy() 是浅拷贝，只复制外层列表，内层列表仍是引用。

示例：

```
import copy
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
# 浅拷贝
list2[0].append(5)
print(list1)
# [[1, 2, 5], [3, 4]]
print(list2)
# [[1, 2, 5], [3, 4]]
```

知识拓展：浅拷贝的影响范围

```
import copy
# 一层嵌套
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
print("修改内层列表：")
list2[0].append(999)
print(f"list1: {list1}")
# [[1, 2, 999], [3, 4]]
print(f"list2: {list2}")
# [[1, 2, 999], [3, 4]]
print("\n替换外层元素：")
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
list2[0] = [999]
print(f"list1: {list1}")
# [[1, 2], [3, 4]] - 不受影响
print(f"list2: {list2}")
# [[999], [3, 4]]
print("\n使用深拷贝：")
list1 = [[1, 2], [3, 4]]
list2 = copy.deepcopy(list1)
list2[0].append(999)
```

```
print(f"list1: {list1}")
# [[1, 2], [3, 4]] - 不受影响
print(f"list2: {list2}")
# [[1, 2, 999], [3, 4]]
```

11 • 单项选择

关于Python的id()函数，以下说法正确的是：

A. 返回对象的值

B. 返回对象的内存地址

C. 返回对象的类型

D. 返回对象的大小

答案 B. 返回对象的内存地址

解析 id() 函数返回对象的唯一标识符，在CPython中是对象的内存地址。

示例：

```
a = [1, 2, 3]
print(id(a))
# 对象的内存地址
b = a
print(id(b))
# 与a相同
c = a.copy()
print(id(c))
# 不同的地址
知识拓展：id()与对象标识
# 相同对象
x = 100
y = 100
print(x is y)
# True（小整数缓存）
print(id(x) == id(y))
# True
# 不同对象
a = [1, 2]
b = [1, 2]
print(a == b)
# True（值相等）
print(a is b)
# False（不是同一对象）
print(id(a) == id(b))
# False
```

12 • 单项选择

以下代码的输出是什么？

```
a = [1, 2, 3]
b = a
c = a.copy()
print(id(a) == id(b), id(a) == id(c))
```

A. True True

B. True False
C. False True
D. False False

答案 B. True False

解析 b是a的引用，id相同；c是a的副本，id不同。
详细示例请参考前面题目的引用与拷贝部分。

13 • 单项选择

关于字符串的不可变性，以下说法正确的是：

A. 字符串可以修改单个字符
B. 字符串是不可变对象
C. 字符串可以使用append()
D. 字符串和列表相同

答案 B. 字符串是不可变对象

解析 字符串是Python中的不可变对象，不能修改其内容。
详细示例请参考前面题目的不可变对象部分。

14 • 单项选择

以下代码的输出是什么？

```
s = "hello"
s[0] = "H"
```

A. "Hello"
B. "hello"
C. 报错TypeError
D. None

答案 C. 报错TypeError

解析 字符串不可变，尝试修改单个字符会引发TypeError。
详细示例请参考前面题目的字符串不可变性部分。

15 • 单项选择

关于函数参数传递，以下说法正确的是：

A. Python使用值传递
B. Python使用引用传递
C. Python使用对象引用传递
D. 取决于参数类型

答案 C. Python使用对象引用传递

解析 Python使用对象引用传递，传递的是对象的引用。
详细示例请参考前面题目的参数传递部分。

16 • 单项选择

以下代码的输出是什么？

```
def change_value(x):
    x = 10
a = 5
```

```
change_value(a)
print(a)
```

A. 5
B. 10
C. None
D. 报错

答案 A. 5

解析 整数是不可变对象，函数内重新赋值不影响外部变量。
详细示例请参考前面题目的不可变对象在函数中的行为部分。

17 • 单项选择

关于列表的extend()和append()方法，以下说法正确的是：

A. 两者完全相同
B. extend()添加多个元素，append()添加单个元素
C. extend()不修改原列表
D. append()返回新列表

答案 B. extend()添加多个元素，append()添加单个元素

解析 extend()添加多个元素，append()添加单个元素（作为整体）。
详细示例请参考第1-5套相关题目。

18 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
lst.extend([4, 5])
print(lst)
```

A. [1, 2, 3, [4, 5]]
B. [1, 2, 3, 4, 5]
C. 1, 2, 3], [4, 5
D. 报错

答案 B. [1, 2, 3, 4, 5]

解析 extend()将可迭代对象的元素逐个添加到列表中。
详细示例请参考第1-5套相关题目。

19 • 单项选择

关于元组的创建，以下哪个是正确的单元素元组？

A. (1)
B. (1,)
C. [1]
D. {1}

答案 B. (1,)

解析 单元素元组需要逗号：(1,)。
详细示例请参考第1-5套相关题目。

20 • 单项选择

以下代码的输出是什么？

```
t = (1)
print(type(t))
```

A. <class 'tuple'>
B. <class 'int'>
C. <class 'list'>
D. 报错

答案 B. <class 'int'>

解析 (1)是整数，不是元组。
详细示例请参考第1-5套相关题目。

21 • 单项选择

Pandas中，DataFrame的可变性体现在：

A. 可以修改单元格的值
B. 可以添加或删除行列
C. 可以修改索引
D. 以上都正确

答案 D. 以上都正确

解析 DataFrame是可变对象，所有选项都正确。
详细示例请参考第1-5套Pandas相关题目。

22 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
df2 = df
df2['B'] = [4, 5, 6]
print('B' in df.columns)
```

A. False
B. True
C. None
D. 报错

答案 B. True

解析 df2是df的引用，修改df2会影响df。
详细示例请参考前面题目的引用部分。

23 • 单项选择

关于字典的可变性，以下说法正确的是：

A. 字典是不可变对象
B. 字典是可变对象
C. 字典的键必须是可变对象
D. 字典的值必须是不可变对象

答案 B. 字典是可变对象

解析 字典是可变对象。
详细示例请参考第1-5套相关题目。

24 • 单项选择

以下代码的输出是什么？

```
d = {'a': [1, 2]}
d['a'].append(3)
print(d)
```

A. {'a': [1, 2]}
B. {'a': [1, 2, 3]}
C. {'a': 3}
D. 报错

答案 B. {'a': [1, 2, 3]}

解析 字典的值是可变列表，可以修改列表内容。
详细示例请参考前面题目的嵌套可变对象部分。

25 • 单项选择

关于集合的可变性，以下说法正确的是：

A. set是可变的，frozenset是不可变的
B. set是不可变的
C. 两者都可变
D. 两者都不可变

答案 A. set是可变的，frozenset是不可变的

解析 set是可变的，frozenset是不可变的。
详细示例请参考第1-5套相关题目。

26 • 单项选择

以下代码的输出是什么？

```
s = {1, 2, 3}
s.add(4)
print(s)
```

A. {1, 2, 3}
B. {1, 2, 3, 4}
C. {4}
D. 报错

答案 B. {1, 2, 3, 4}

解析 set.add()方法添加元素。
详细示例请参考第1-5套相关题目。

27 • 单项选择

关于列表推导式中的可变性，以下说法正确的是：

A. 列表推导式创建新列表
B. 列表推导式修改原列表
C. 列表推导式返回元组
D. 列表推导式不能使用

答案 A. 列表推导式创建新列表

解析 列表推导式创建新列表，不修改原列表。

详细示例请参考第1-5套相关题目。

28 • 单项选择

以下代码的输出是什么？

```
original = [1, 2, 3]
new = [x * 2 for x in original]
print(original)
```

A. [2, 4, 6]

B. [1, 2, 3]

C. [1, 2, 3, 2, 4, 6]

D. 报错

答案 B. [1, 2, 3]

解析 列表推导式不修改原列表。

详细示例请参考第1-5套相关题目。

29 • 单项选择

关于函数返回可变对象，以下说法正确的是：

A. 返回的是对象的引用

B. 返回的是对象的副本

C. 返回的是新对象

D. 不能返回可变对象

答案 A. 返回的是对象的引用

解析 函数返回可变对象时，返回的是对象的引用。

详细示例请参考前面题目的引用部分。

30 • 单项选择

以下代码的输出是什么？

```
def get_list():
    return [1, 2, 3]
lst1 = get_list()
lst2 = get_list()
print(id(lst1) == id(lst2))
```

A. True

B. False

C. None

D. 报错

答案 B. False

解析 每次调用函数都创建新列表，id不同。

详细示例请参考前面题目的对象创建部分。

二、不定项选择题

31 • 不定项选择

关于Python的可变对象，以下说法正确的有：

A. 列表是可变对象
B. 字典是可变对象
C. 集合是可变对象
D. 字符串是可变对象

答案 A. 列表是可变对象；B. 字典是可变对象；C. 集合是可变对象

解析 字符串是不可变对象，列表、字典、集合是可变对象。
（详细示例请参考前面单选题的可变对象部分）

32 • 不定项选择

关于Python的不可变对象，以下说法正确的有：

A. 元组是不可变对象
B. 字符串是不可变对象
C. 整数是不可变对象
D. frozenset是不可变对象

答案 A. 元组是不可变对象；B. 字符串是不可变对象；C. 整数是不可变对象；D. frozenset是不可变对象

解析 所有选项都是不可变对象。
（详细示例请参考前面单选题的不可变对象部分）

33 • 不定项选择

以下哪些操作会修改原列表：

A. `lst.append(x)`
B. `lst.extend(iterable)`
C. `lst.sort()`
D. `sorted(lst)`

答案 A. `lst.append(x)`；B. `lst.extend(iterable)`；C. `lst.sort()`

解析 `sorted()` 返回新列表，不修改原列表。
（详细示例请参考第1-5套相关题目）

34 • 不定项选择

关于可变对象作为默认参数的问题，以下说法正确的有：

A. 默认参数在函数定义时创建
B. 每次调用函数都会创建新的默认参数
C. 可变默认参数会在多次调用间共享
D. 应该使用None作为默认值

答案 A. 默认参数在函数定义时创建；C. 可变默认参数会在多次调用间共享；D. 应该使用None作为默认值

解析 默认参数在函数定义时创建一次，每次调用不会创建新的默认参数。
（详细示例请参考前面单选题的默认参数陷阱部分）

35 • 不定项选择

以下哪些方法可以创建列表的副本：

A. `lst.copy()`
B. `lst[:]`

C. list(lst)

D. copy.deepcopy(lst)

答案 A. lst.copy()；B. lst[:]；C. list(lst)；D. copy.deepcopy(lst)

解析 所有方法都可以创建列表的副本。

（详细示例请参考前面单选题的浅拷贝部分）

36 • 不定项选择

关于元组的特性，以下说法正确的有：

A. 元组不可变是指不能修改元组的元素引用

B. 元组中的可变对象可以被修改

C. 元组不能作为字典的键

D. 元组比列表占用更少内存

答案 A. 元组不可变是指不能修改元组的元素引用；B. 元组中的可变对象可以被修改；D. 元组比列表占用更少内存

解析 包含不可变元素的元组可以作为字典的键。

（详细示例请参考前面单选题的元组特性部分）

37 • 不定项选择

关于深拷贝和浅拷贝，以下说法正确的有：

A. 浅拷贝只复制对象本身

B. 深拷贝递归复制所有嵌套对象

C. copy.copy() 是浅拷贝

D. copy.deepcopy() 是深拷贝

答案 A. 浅拷贝只复制对象本身；B. 深拷贝递归复制所有嵌套对象；C. copy.copy() 是浅拷贝；D. copy.deepcopy() 是深拷贝

解析 所有关于深拷贝和浅拷贝的选项都正确。

（详细示例请参考前面单选题的拷贝部分）

38 • 不定项选择

以下哪些操作不会修改原字符串：

A. s.upper()

B. s.replace('a', 'b')

C. s.split()

D. 所有字符串方法都不修改原字符串

答案 D. 所有字符串方法都不修改原字符串

解析 所有字符串方法都返回新字符串，不修改原字符串。

（详细示例请参考前面单选题的字符串不可变性部分）

39 • 不定项选择

关于函数参数的可变性，以下说法正确的有：

A. 传递不可变对象时，函数内修改不影响外部

B. 传递可变对象时，函数内修改会影响外部

C. 可以通过重新赋值避免修改外部对象

D. 应该在函数内创建副本来避免修改

答案 A. 传递不可变对象时，函数内修改不影响外部；B. 传递可变对象时，函数内修改会影响外部；C. 可以通过重新赋值避免修改外部对象；D. 应该在函数内创建副本来避免修改

解析 所有关于函数参数可变性的选项都正确。
（详细示例请参考前面单选题的参数传递部分）

40 • 不定项选择

关于列表的+=和extend()，以下说法正确的有：

A. lst += [1, 2] 会修改原列表
B. lst = lst + [1, 2] 创建新列表
C. lst.extend([1, 2]) 修改原列表
D. 三者效果完全相同

答案 A. lst += [1, 2] 会修改原列表；B. lst = lst + [1, 2] 创建新列表；C. lst.extend([1, 2]) 修改原列表

解析 += 修改原列表， + 创建新列表， extend() 修改原列表。
（详细示例请参考第1-5套相关题目）

41 • 不定项选择

关于Pandas中的视图和副本，以下说法正确的有：

A. 某些操作返回视图
B. 某些操作返回副本
C. 修改视图会影响原DataFrame
D. 使用 .copy() 可以创建副本

答案 A. 某些操作返回视图；B. 某些操作返回副本；C. 修改视图会影响原DataFrame；D. 使用 .copy() 可以创建副本

解析 所有关于Pandas视图和副本的选项都正确。
（详细示例请参考第1-5套Pandas相关题目）

42 • 不定项选择

关于字典的可变性，以下说法正确的有：

A. 字典的键必须是不可变对象
B. 字典的值可以是任何对象
C. 可以动态添加键值对
D. 可以删除键值对

答案 A. 字典的键必须是不可变对象；B. 字典的值可以是任何对象；C. 可以动态添加键值对；D. 可以删除键值对

解析 所有选项都正确。
（详细示例请参考第1-5套相关题目）

43 • 不定项选择

关于对象的引用计数，以下说法正确的有：

A. Python使用引用计数管理内存
B. 赋值操作增加引用计数
C. del语句减少引用计数
D. 引用计数为0时对象被回收

答案 A. Python使用引用计数管理内存；B. 赋值操作增加引用计数；C. del语句减少引用计数；D. 引用计数为0时对象被回收

解析 所有选项都正确。
（详细示例请参考第1-5套相关题目）

44 • 不定项选择

以下哪些是避免可变默认参数问题的方法：

A. 使用None作为默认值
B. 使用元组代替列表
C. 在函数内部创建新对象
D. 使用不可变对象作为默认值

答案 A. 使用None作为默认值；B. 使用元组代替列表；C. 在函数内部创建新对象；D. 使用不可变对象作为默认值

解析 所有选项都是正确的方法。
（详细示例请参考第1-5套相关题目）

45 • 不定项选择

关于列表的切片操作，以下说法正确的有：

A. 切片返回新列表
B. 切片不修改原列表
C. `lst[:]` 创建浅拷贝
D. 切片可以用于赋值

答案 A. 切片返回新列表；B. 切片不修改原列表；C. `lst[:]` 创建浅拷贝；D. 切片可以用于赋值

解析 所有选项都正确。
（详细示例请参考第1-5套相关题目）

三、填空题

46 • 填空 / 简答

列表是 _____ 对象，元组是 _____ 对象。

答案 可变；不可变

解析 （原卷第6套填空答案缺失/错位，依据题目整理）列表 `list` 是可变对象，元组 `tuple` 是不可变对象。

47 • 填空 / 简答

可变对象作为函数默认参数时，默认参数在 _____ 时创建，多次调用会 _____ 同一个对象。

答案 函数定义（def 执行）时；共享（指向）同一个对象

解析 （原卷答案缺失，依据题目整理）默认参数只在函数定义时求值一次，之后每次调用都复用同一个对象。

48 • 填空 / 简答

使用 _____ 模块的 _____ 函数可以创建对象的深拷贝。

答案 copy；deepcopy

解析 （原卷答案错位，依据题目整理）`copy.deepcopy()` 递归复制所有嵌套对象。

49 • 填空 / 简答

在函数中修改可变参数时，如果不想影响外部对象，应该先创建 _____，或者使用 _____ 方法。

答案 副本（拷贝）；copy()（或 deepcopy()）

解析 （原卷答案缺失，依据题目整理）先复制再修改，避免原地修改外部传入的可变对象。

50 • 填空 / 简答

元组的不可变性是指 _____ 不可变，但元组中的 _____ 对象仍然可以被修改。

答案 元组本身的元素引用；可变

解析 （原卷答案缺失，依据题目整理）元组不可变指元素引用不能换，但元素本身若是可变对象仍可改。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
def add_item(item, items=[]):
    items.append(item)
    return items
list1 = add_item(1)
list2 = add_item(2)
list3 = add_item(3, [])
print(list1)
print(list2)
print(list3)
```

问题：

- (1) 三个print语句的输出分别是什么？
- (2) 为什么list1和list2的结果不是预期的？
- (3) 如何修改函数避免这个问题？

答案 （1）[1, 2]；[1, 2]；[3]
（2）默认参数 items=[] 在函数定义时只创建一次，list1 和 list2 指向同一个列表，第二次调用继续在同一个列表上 append。
（3）改为 items=None，函数内 if items is None: items = []。

解析 （原卷答案缺失，依据题目整理）

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def modify_data(data):
    data.append(4)
    data = [1, 2, 3]
    data.append(5)
    return data
original = [0]
result = modify_data(original)
```

```
print(original)
print(result)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么original被修改了？
- (3) 重新赋值 data = [1, 2, 3] 后为什么不影响original？

答案 （1）[0, 4]；[1, 2, 3, 5]
（2）data.append(4) 原地修改了 original 指向的列表。
（3）data = [1, 2, 3] 只是让 data 重新指向新列表，不再指向 original。

解析 （原卷答案缺失，依据题目整理）

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import copy
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
list3 = copy.deepcopy(list1)
list1[0].append(5)
list2[1].append(6)
print(list1)
print(list2)
print(list3)
```

问题：

- (1) 三个print语句的输出分别是什么？
- (2) 浅拷贝和深拷贝的区别是什么？
- (3) 什么时候应该使用深拷贝？

答案 （1）[[1, 2, 5], [3, 4, 6]]；[[1, 2, 5], [3, 4, 6]]；[[1, 2], [3, 4]]
（2）浅拷贝只复制外层，内层仍是同一对象；深拷贝递归复制所有层。
（3）对象含嵌套可变对象、且需要完全独立的副本时用 deepcopy。

解析 （原卷答案缺失，依据题目整理）

54 • 填空 / 简答

阅读以下代码，回答问题：

```
t = ([1, 2], 3)
t[0].append(4)
print(t)
try:
    t[1] = 5
except TypeError as e:
    print("错误:", e)
```

问题：

- (1) 第一个print的输出是什么？
- (2) 为什么可以修改t[0]但不能修改t[1]？
- (3) 这说明了元组的什么特性？

答案 （1）([1, 2, 4], 3)
（2）t[0] 是列表，列表本身可变；t[1] 是元组元素引用，元组不允许重新赋值。
（3）元组的不可变性指元素引用固定，元素对象本身可以是可变的。

解析 （原卷答案缺失，依据题目整理）

55 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df1 = pd.DataFrame({'A': [1, 2, 3]})
df2 = df1
df3 = df1.copy()
df1['B'] = [4, 5, 6]
print('B' in df2.columns)
print('B' in df3.columns)
print(id(df1) == id(df2))
```

问题：

- (1) 三个print语句的输出分别是什么？
- (2) df2和df3有什么区别？
- (3) 在Pandas中什么时候需要使用copy()？

答案 （1）True；False；True
（2）df2 与 df1 是同一个对象（别名）；df3 是独立副本。
（3）需要在修改数据时不污染原 DataFrame（尤其是切片/筛选后）时使用 copy()。

解析 （原卷答案缺失，依据题目整理）

五、程序设计题

56 • 填空 / 简答

设计一个 SafeList 类，要求：

1. 继承自
   继承自list
2. 重写修改方法
   重写修改方法：
   - 所有修改操作（append、extend、insert等）都记录日志
   - 提供 undo() 方法撤销最后一次修改
   - 提供 get_history() 查看修改历史
3. 实现深拷贝方法

实现深拷贝方法：

- deep_copy() 返回完全独立的副本
- shallow_copy() 返回浅拷贝

4. 防御性编程

防御性编程：

- 提供 safe_append(item, max_size) 限制列表大小
- 提供 freeze() 方法使列表变为只读

要求：

- 使用装饰器记录操作

- 处理嵌套列表的情况
- 提供清晰的错误提示

答案

```
import copy
from functools import wraps
def log_operation(func):
    @wraps(func)
    def wrapper(self, *args, **kwargs):
        # 保存操作前的状态
        state = copy.deepcopy(list(self))
        self._history.append(('before', state, func.__name__))
        # 执行操作
        result = func(self, *args, **kwargs)
        # 保存操作后的状态
        state = copy.deepcopy(list(self))
        self._history.append(('after', state, func.__name__))
        print(f"操作: {func.__name__}, 参数: {args}")
        return result
    return wrapper
class SafeList(list):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._history = []
        self._frozen = False
    def _check_frozen(self):
        if self._frozen:
            raise RuntimeError("列表已被冻结，不能修改")
    @log_operation
    def append(self, item):
        self._check_frozen()
        super().append(item)
    @log_operation
    def extend(self, iterable):
        self._check_frozen()
    super().extend(iterable)
@log_operation
def insert(self, index, item):
    self._check_frozen()
    super().insert(index, item)
@log_operation
def remove(self, item):
    self._check_frozen()
    super().remove(item)
@log_operation
def pop(self, index=-1):
    self._check_frozen()
    return super().pop(index)
def undo(self):
```

```
    if len(self._history) < 2:
        print("没有可撤销的操作")
        return
    # 找到最后一个'before'状态
    for i in range(len(self._history) - 1, -1, -1):
        if self._history[i][0] == 'before':
            state = self._history[i][1]
            self.clear()
            self.extend(state)
            print(f"撤销操作: {self._history[i][2]}")
            break
def get_history(self):
    return [(op, name) for op, _, name in self._history]
def deep_copy(self):
    return copy.deepcopy(self)
def shallow_copy(self):
    return SafeList(self)
    def safe_append(self, item, max_size=None):
        if max_size and len(self) >= max_size:
            raise ValueError(f"列表大小已达到限制: {max_size}")
        self.append(item)
    def freeze(self):
        self._frozen = True
        print("列表已冻结")
    def unfreeze(self):
        self._frozen = False
        print("列表已解冻")
# 使用示例
sl = SafeList([1, 2, 3])
sl.append(4)
sl.extend([5, 6])
print(f"当前列表: {sl}")
print(f"\n操作历史: {sl.get_history()}")
sl.undo()
print(f"撤销后: {sl}")
# 测试深拷贝
sl2 = sl.deep_copy()
sl2.append(100)
print(f"\n原列表: {sl}")
print(f"深拷贝: {sl2}")
# 测试冻结
sl.freeze()
try:
    sl.append(7)
except RuntimeError as e:
    print(f"\n错误: {e}")
```

解析

57 • 填空 / 简答

编写装饰器和工具函数处理可变参数问题：

1. 装饰器

装饰器 @safe_mutable_default ：

- 自动处理可变默认参数
- 每次调用创建新的默认对象
- 支持list、dict、set等

2. 装饰器

装饰器 @immutable_params ：

- 确保函数不修改传入的可变参数
- 在函数内部自动创建副本
- 修改尝试时给出警告

3. 工具函数

工具函数：

- safe_copy(obj, deep=False) ：智能拷贝
- is_mutable(obj) ：判断对象是否可变
- freeze_object(obj) ：将可变对象转为不可变

演示这些工具的使用场景。

答案

```
import copy
from functools import wraps
import sys
def safe_mutable_default(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        # 获取函数签名
        import inspect
        sig = inspect.signature(func)
        bound = sig.bind_partial(*args, **kwargs)
        # 处理默认参数
        for param_name, param in sig.parameters.items():
            if param_name not in bound.arguments and param.default is not param.empty:
                if isinstance(param.default, (list, dict, set)):
                # 创建新的可变对象
                if isinstance(param.default, list):
                bound.arguments[param_name] = []
                elif isinstance(param.default, dict):
                bound.arguments[param_name] = {}
                elif isinstance(param.default, set):
                bound.arguments[param_name] = set()
        return func(*bound.args, **bound.kwargs)
    return wrapper
def immutable_params(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        # 创建参数的副本
        new_args = []
        for arg in args:
```

```
            if is_mutable(arg):
                new_args.append(copy.deepcopy(arg))
            else:
                new_args.append(arg)
        new_kwargs = {}
        for key, value in kwargs.items():
            if is_mutable(value):
                new_kwargs[key] = copy.deepcopy(value)
            else:
                new_kwargs[key] = value
        return func(*new_args, **new_kwargs)
    return wrapper
def safe_copy(obj, deep=False):
    """智能拷贝对象"""
    if not is_mutable(obj):
        return obj
    if deep:
        return copy.deepcopy(obj)
    else:
        if isinstance(obj, list):
            return obj.copy()
        elif isinstance(obj, dict):
            return obj.copy()
        elif isinstance(obj, set):
            return obj.copy()
        else:
            return copy.copy(obj)
def is_mutable(obj):
    """判断对象是否可变"""
    immutable_types = (int, float, str, tuple, frozenset, bytes, bool, type(None))
    return not isinstance(obj, immutable_types)
def freeze_object(obj):
    """将可变对象转为不可变对象"""
    if isinstance(obj, list):
        return tuple(freeze_object(item) for item in obj)
    elif isinstance(obj, dict):
        return tuple((k, freeze_object(v)) for k, v in sorted(obj.items()))
    elif isinstance(obj, set):
        return frozenset(freeze_object(item) for item in obj)
    else:
        return obj
# 使用示例
print("=== safe_mutable_default 示例 ===")
@safe_mutable_default
def add_to_list(item, lst=[]):
    lst.append(item)
    return lst
print(add_to_list(1))
```

```
# [1]
print(add_to_list(2))
# [2] 而不是 [1, 2]
print("\n=== immutable_params 示例 ===")
@immutable_params
def modify_list(lst):
    lst.append(999)
    return lst
original = [1, 2, 3]
result = modify_list(original)
print(f"原列表: {original}")
# [1, 2, 3]
print(f"返回值: {result}")
# [1, 2, 3, 999]
print("\n=== 工具函数示例 ===")
data = {'a': [1, 2], 'b': {'c': 3}}
print(f"是否可变: {is_mutable(data)}")
frozen = freeze_object(data)
print(f"冻结后: {frozen}")
```

解析

58 • 填空 / 简答

使用Pandas和Python内置类型，实现数据容器分析：

1. 分析函数

分析函数：

- analyze_mutability(obj) ：分析对象的可变性
- find_mutable_elements(container) ：查找容器中的可变元素
- calculate_memory_usage(obj) ：计算对象内存占用

2. 对比测试

对比测试：

- 列表vs元组的性能对比
- 浅拷贝vs深拷贝的性能对比
- 不同数据结构的内存占用对比

3. 生成报告

生成报告：

- 使用Pandas创建对比表格
- 可视化内存和性能差异
- 给出优化建议

要求：

- 使用装饰器测量性能
- 使用生成器处理大数据
- 导出分析报告

答案

```
import sys
import time
import pandas as pd
from functools import wraps
import matplotlib.pyplot as plt
```

```
def measure_performance(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        return result, end - start
    return wrapper
def analyze_mutability(obj):
    """分析对象的可变性"""
    info = {
        'type': type(obj).__name__,
        'is_mutable': is_mutable(obj),
        'size': sys.getsizeof(obj),
        'id': id(obj)
    }
    if isinstance(obj, (list, tuple)):
        info['length'] = len(obj)
        info['has_mutable_elements'] = any(is_mutable(item) for item in obj)
    elif isinstance(obj, dict):
        info['length'] = len(obj)
        info['has_mutable_values'] = any(is_mutable(v) for v in obj.values())
    return info
def find_mutable_elements(container):
    """查找容器中的可变元素"""
    mutable_items = []
    if isinstance(container, (list, tuple)):
        for i, item in enumerate(container):
            if is_mutable(item):
                mutable_items.append((i, type(item).__name__, item))
    elif isinstance(container, dict):
        for key, value in container.items():
            if is_mutable(value):
                mutable_items.append((key, type(value).__name__, value))
    return mutable_items
def calculate_memory_usage(obj):
    """计算对象内存占用"""
    total = sys.getsizeof(obj)
    if isinstance(obj, (list, tuple)):
        for item in obj:
            total += sys.getsizeof(item)
    elif isinstance(obj, dict):
        for key, value in obj.items():
            total += sys.getsizeof(key) + sys.getsizeof(value)
    return total
def is_mutable(obj):
    immutable_types = (int, float, str, tuple, frozenset, bytes, bool, type(None))
    return not isinstance(obj, immutable_types)
```

```
@measure_performance
def test_list_operations(n):
    lst = []
    for i in range(n):
        lst.append(i)
    return lst
@measure_performance
def test_tuple_creation(n):
    return tuple(range(n))
@measure_performance
def test_shallow_copy(obj):
    return obj.copy()
@measure_performance
def test_deep_copy(obj):
    import copy
    return copy.deepcopy(obj)
# 性能对比测试
def performance_comparison():
    sizes = [100, 1000, 10000]
    results = []
    for size in sizes:
        # 列表 vs 元组
        list_result, list_time = test_list_operations(size)
        tuple_result, tuple_time = test_tuple_creation(size)
        # 浅拷贝 vs 深拷贝
        nested_list = [[i] for i in range(size)]
        _, shallow_time = test_shallow_copy(nested_list)
        _, deep_time = test_deep_copy(nested_list)
        results.append({
            'size': size,
            'list_time': list_time,
            'tuple_time': tuple_time,
            'shallow_copy_time': shallow_time,
            'deep_copy_time': deep_time,
            'list_memory': sys.getsizeof(list_result),
            'tuple_memory': sys.getsizeof(tuple_result)
        })
    return pd.DataFrame(results)
# 生成分析报告
def generate_report():
    print("=== 数据容器分析报告 ===\n")
    # 分析不同对象
    test_objects = {
        '列表': [1, 2, 3, [4, 5]],
    '元组': (1, 2, 3, (4, 5)),
    '字典': {'a': [1, 2], 'b': 3},
    '集合': {1, 2, 3}
}
```

```
print("1. 对象可变性分析:")
for name, obj in test_objects.items():
    info = analyze_mutability(obj)
    print(f"\n{name}:")
    for key, value in info.items():
        print(f"
                {key}: {value}")
# 查找可变元素
print("\n\n2. 可变元素检测:")
container = [1, "hello", [2, 3], {'a': 1}, (4, 5)]
mutable = find_mutable_elements(container)
for idx, type_name, item in mutable:
    print(f" 索引{idx}: {type_name} - {item}")
# 性能对比
print("\n\n3. 性能对比:")
df = performance_comparison()
print(df.to_string(index=False))
# 保存报告
df.to_excel('container_analysis_report.xlsx', index=False)
print("\n报告已保存到 container_analysis_report.xlsx")
# 可视化
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
# 时间对比
df.plot(x='size', y=['list_time', 'tuple_time'], ax=axes[0], marker='o')
axes[0].set_title('列表 vs 元组创建时间')
axes[0].set_xlabel('元素数量')
axes[0].set_ylabel('时间(秒)')
# 拷贝时间对比
df.plot(x='size', y=['shallow_copy_time', 'deep_copy_time'], ax=axes[1], marker='o')
axes[1].set_title('浅拷贝 vs 深拷贝时间')
axes[1].set_xlabel('元素数量')
axes[1].set_ylabel('时间(秒)')
plt.tight_layout()
plt.savefig('performance_comparison.png')
print("性能对比图已保存到 performance_comparison.png")
# 运行分析
generate_report()
```

题目4和5的参考答案

的参考答案由于篇幅限制，核心思路：

解析

59 • 填空 / 简答

实现一个配置管理系统，处理可变性问题：

1. Config类：

- 使用不可变对象存储配置
- 提供 get() 、 set() 方法
- 配置变更时创建新版本（不修改旧版本）

2. ConfigHistory类：
• 记录所有配置变更
• 支持回滚到历史版本
• 使用深拷贝保存每个版本
3. 功能要求
功能要求：
• 支持嵌套配置
• 配置验证（类型检查）
• 配置导入导出（JSON/YAML）
• 配置差异对比
要求：
• 使用 pathlib 处理文件
• 确保线程安全
• 实现配置继承

答案 ： 配置管理系统- 使用不可变数据结构（如frozendict或自定义类），每次修改创建新版本，使用深拷贝保存历史。

解析

60 • 填空 / 简答

实现各种数据结构之间的安全转换：
1. 转换函数
转换函数：
• list_to_tuple(lst, deep=False) ：列表转元组
• tuple_to_list(tpl, deep=False) ：元组转列表
• dict_to_frozendict(d) ：字典转不可变字典
• mutable_to_immutable(obj) ：通用转换
2. 验证函数
验证函数：
• validate_immutable(obj) ：验证对象完全不可变
• find_mutable_refs(obj) ：查找可变引用
• ensure_deep_immutable(obj) ：确保深度不可变
3. 应用场景
应用场景：
• 将配置数据转为不可变对象
• 缓存键的安全处理
• 函数参数的防御性复制
要求：
• 处理循环引用
• 支持自定义类型
• 提供详细的转换报告

答案 ： 数据结构转换工具- 递归处理嵌套结构，检测循环引用，提供完整的可变/不可变转换功能。
评分说明：
• 单选题：每题1分，共30分
• 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
• 填空题：每空1分，共10分

- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析